SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL

1

Analytical Problem Solving

Name of the Student	**P.Mokshith Reddy**
Reg. No	**192425149**

COURSE NAME: Deep Learning **COURSE CODE: MLA0403 FACULTY : Dr.Arul Raj A.M**

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 25

Code of Conduct:9

I P.Mokshith Reddy/ 192425149 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **Name of the student:**

Mokshith **P.MOKSHITH REDDY**

QUESTIONS:05 **Total Marks:25**

Analytical Problem Solving

1. Learning Algorithms (Optimization Behavior)

A machine learning model is trained using gradient descent on a convex loss function. Initially, the learning rate is set very high, and later it is gradually decreased.

Question:
Analyze how the choice of a high initial learning rate followed by decay affects convergence. Under what conditions can this strategy outperform a constant learning rate? Discuss possible risks and justify with reasoning.
Anser :
Gradient Descent updates the model parameters using:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta)_t$$

where:
$\theta = model\ parameters$ $\eta =$
$Learning\ Rate$
$J(\theta) = Loss\ Function$

A **high initial learning rate** allows the optimizer to take large steps toward the global minimum, enabling faster progress during the early stages of training. Since the loss function is convex, there is only one global optimum, making rapid exploration beneficial.

As training progresses, **learning rate decay** gradually reduces the step size. Smaller updates help the optimizer converge smoothly and avoid oscillations around the minimum.

Advantages over a Constant Learning Rate
A decaying learning rate performs better when:

- The initial parameters are far from the optimum.
- The optimization landscape is smooth and convex.
- Fast exploration is required initially, followed by precise convergence.

Compared to a constant learning rate:
- Faster initial convergence.
- Better final accuracy.
- Reduced oscillations near the optimum.

Risks
- If the initial learning rate is too high, the algorithm may overshoot the minimum.
- Large updates can cause unstable training or divergence.
- If the learning rate decays too quickly, training may stop before reaching the optimal solution.

Conclusion
A high initial learning rate followed by gradual decay provides a balance between **fast optimization** and **stable convergence**, often outperforming a constant learning rate when properly tuned.

2. Maximum Likelihood Estimation (MLE)

Suppose you are given a dataset generated from a Gaussian distribution with unknown mean μ and known variance σ^2

Question:

Derive the Maximum Likelihood Estimator for μ, and analytically explain how the estimate changes when:

- The dataset contains outliers
- The sample size increases
  What does this imply about the robustness of MLE?

Answers:

For observations

$$x_1, x_2, \ldots, x_n \sim N(\mu, \sigma^2)$$

where variance σ2 is known.

The likelihood is :

$$L(\mu) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x_i-\mu)^2}{2\sigma^2}}$$

Taking the log-likelihood, :

$$l(\mu) = -\frac{n}{2}\ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i - \mu)^2$$

Differentiate with respect to μ\muμ:

$$\frac{dl}{d\mu} = \frac{1}{\sigma^2}\sum_{i=1}^{n}(x_i - \mu)$$

Setting the derivative equal to zero,

$$\mu\hat{} =$$

$$n1i=1\sum nx$$

$$i\ \mu\hat{}=\bar{x}$$

Effect of Outliers

Since the estimator is the sample mean, extreme values directly influence the estimate. A few outliers can significantly shift the estimated mean away from the true value.

Effect of Increasing Sample Size

As the sample size increases,

- the estimate becomes more stable,
- variance decreases,

- the estimator approaches the true mean (Law of Large Numbers).

Implication

MLE is:

- **Efficient and consistent** for large datasets.
- **Sensitive to outliers**, meaning it is **not robust** because every observation contributes equally to the sample mean.

3. Building a Machine Learning Algorithm (Model Selection)

You are designing a classification algorithm for a dataset with 10,000 features but only 500 training samples.

Question:
Analyze the challenges involved in training such a model. Propose a strategy (algorithm pipeline) to handle this situation and justify each step (e.g., feature selection, regularization, dimensionality reduction). Explain how your approach mitigates overfitting.

Answer:

The dataset contains:

- **10,000 features**
- **500 training samples**

Since the number of features greatly exceeds the number of samples (p>>np>>np>>n), the model suffers from the **curse of dimensionality**.

Challenges

- High risk of overfitting.
- Increased computational complexity.
- Presence of redundant and irrelevant features.
- Poor model generalization.

Proposed Pipeline

Step 1: Data Preprocessing

- Normalize features using StandardScaler.
- Ensures equal feature contribution.

Step 2: Feature Selection

- Remove low-variance features.
- Apply LASSO or Mutual Information.
- Keep only informative features.

Step 3: Dimensionality Reduction

- Apply PCA.
- Reduce thousands of correlated features into fewer principal components while preserving most variance.

Step 4: Regularization

- Use Logistic Regression with L2 regularization or Support Vector Machine.
- Penalizes overly complex models.

Step 5: Cross Validation

- Perform k-fold cross-validation.
- Select hyperparameters that generalize well.

Why This Works

Feature selection removes irrelevant variables.

PCA reduces dimensionality.

Regularization limits model complexity.

Cross-validation prevents selecting an overfitted model.

Together, these steps reduce variance and improve

generalization.

4. Neural Networks (Multilayer Perceptron - MLP)

Consider a Multilayer Perceptron with one hidden layer using sigmoid activation functions.

Question:
Explain analytically why adding more hidden neurons increases the representational power of the network. Then, reason why this does not always lead to better performance. Support your answer using concepts like overfitting and generalization.

Answer:

An MLP consists of an input layer, hidden layer(s), and an output layer.

Each hidden neuron computes

a=σ(wx+b)a=\sigma(wx+b)a=σ(wx+b)

where σ\sigmaσ is the sigmoid activation.

Why More Hidden Neurons Increase Representational Power

Each neuron learns a different nonlinear feature.

Adding more neurons enables the network to:

- learn more complex decision boundaries,
- approximate highly nonlinear functions,
- represent complicated input-output relationships.

According to the **Universal Approximation Theorem**, an MLP with enough hidden neurons can approximate any continuous function.

Why More Neurons Do Not Always Improve Performance

Increasing hidden neurons also increases:

- number of trainable parameters,
- model complexity,
- computational cost.

With limited training data, the network may memorize training examples instead of learning general patterns.

This causes **overfitting**.

Training accuracy becomes high, while testing accuracy decreases.

Generalization

A model should capture underlying patterns rather than memorize the training data.

Therefore,

- too few neurons → underfitting,
- too many neurons → overfitting,
- an optimal number provides the best generalization.

5. Backpropagation, SGD & Curse of Dimensionality

A deep neural network is trained using Stochastic Gradient Descent (SGD) on a very high-dimensional dataset.

Question:
Analyze how the **curse of dimensionality** impacts:

- Gradient updates in backpropagation
- Convergence speed of SGD
- Model generalization

Propose at least two techniques to overcome these issues and justify them analytically.

Answer:

The **curse of dimensionality** refers to the problems caused when the number of input features becomes very large.

Impact on Gradient Updates

High-dimensional parameter spaces create:

- noisy gradients,
- sparse data,
- unstable parameter updates.

Backpropagation becomes less efficient because gradients carry less useful information.

Impact on SGD Convergence

SGD already uses noisy gradient estimates.

In high dimensions:

- optimization requires more iterations,
- gradients fluctuate,
- convergence becomes slower.

Impact on Model Generalization

When features greatly exceed training samples:

- the model memorizes noise,
- variance increases,

- prediction accuracy on unseen data decreases.

This leads to poor generalization.

Techniques to Overcome These Problems

1. Dimensionality Reduction (PCA)

PCA transforms high-dimensional data into a lower-dimensional space while preserving maximum variance.

Benefits:

- faster optimization,
- reduced computational cost,
- improved convergence.

2. Regularization

L1 or L2 regularization penalizes excessively large weights.

Benefits:

- prevents overfitting,
- improves model generalization,
- simplifies the learned model.

Additional Techniques

- Feature Selection
- Batch Normalization
- Early Stopping
- Dropout

Conclusion

Reducing dimensionality and controlling model complexity improve gradient quality, accelerate SGD convergence, and enhance the model's ability to generalize to unseen data.